

Lekce 14: Použití baseUrl a kontextů v Cypressu

Cíle

- Využívejte konfiguraci `baseUrl` pro zjednodušení navigace v testech a používání relativních URL.
- Pochopte a strukturalizujte testy pomocí kontextů pro správu různých scénářů.
- Využívejte pokročilé příkazy Cypressu jako `cy.origin` pro práci s více doménami.
- Zavádějte osvědčené postupy pro strukturu a udržitelnost testů.

Rozpis obsahu

A. baseUrl

1. Nastavení baseUrl v konfiguraci Cypressu:

- **Definice:**

`baseUrl` je konfigurační nastavení v Cypressu, které určuje kořenovou URL pro všechny vaše testy. To vám umožní používat v příkazech, jako je `cy.visit('/')`, relativní URL.

- **Příklad:**

```
// cypress.config.js
import { defineConfig } from 'cypress';

export default defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    // Další nastavení...
  },
});
```

2. Výhody používání baseUrl:

- **Zjednodušení:**

Vyhnete se opakování celých URL v testech; místo toho použijete relativní cesty.

- **Flexibilita:**

Snadná změna prostředí přepsáním `baseUrl` (např. vývoj, staging, produkce).

- **Udržitelnost:**

Centralizovaná konfigurace URL; pokud se hostitel změní, stačí aktualizovat jeden soubor.

3. Přepsání baseUrl pro různá prostředí:

- Použijte environmentální proměnné nebo samostatné konfigurační soubory ke změně baseUrl.

- **Příklad:**

```
// V cypress.config.js pomocí loadEnv (viz Lekce 12)
import { defineConfig, loadEnv } from 'vite';
export default defineConfig(({ mode }) => {
  const env = loadEnv(mode, process.cwd(), ['VITE_']);
  return {
    e2e: {
      baseUrl: env.VITE_API_BASE_URL || 'http://localhost:3000',
      // další nastavení...
    }
  };
});
```

- Pomocí různých .env souborů (např. .env.development , .env.production) lze baseUrl přepínat podle potřeby.

B. Kontexty v Cypressu

1. Pochopení kontextů v Cypressu:

- **Definice:**

Kontexty (nebo logické seskupení) se používají k organizaci testů, které sdílejí společné nastavení nebo patří do určitého uživatelského scénáře.

- **Použití:**

I když Cypress nemá speciální funkci pro kontext (protože context() je pouze aliasem pro describe()), můžete použít zanořené bloky describe() k vytvoření smysluplných skupin.

2. Použití kontextů pro správu různých testovacích scénářů:

- **Příklad:**

Seskupte testy na základě různých stavů aplikace nebo toků (např. „když je uživatel přihlášen“, „když uživatel není přihlášen“).

- **Příklad vnořeného describe:**

```

describe('Login Page @login', () => {
  beforeEach(() => {
    cy.visit('/');
  });

  describe('Validation Errors', () => {
    it('should display error when username is missing', () => {
      // Testovací kód
    });
    it('should display error when password is missing', () => {
      // Testovací kód
    });
  });
});

describe('Successful Login', () => {
  it('should log in successfully with valid credentials', () => {
    // Testovací kód
  });
});

```

Ve světě Cypressu obvykle testovací kontext znamená prostředí, ve kterém je test vykonáván. To zahrnuje:

- JavaScriptový scope, ve kterém Cypress provádí váš kód,
- doménu (původ) stránky, kterou navštěvujete,
- stav prohlížeče jako cookies, localStorage, session, a dokonce proměnné v paměti.

Multidoménové testování v Cypressu

Co je multidoménové testování?

Multidoménové testování označuje scénáře, kdy vaše aplikace zahrnuje více než jednu doménu nebo subdoménu. To je běžné například tehdy, když vaše aplikace:

- Používá ověřování třetí strany (např. OAuth poskytovatelé, sociální přihlášení).
- Má oddělené domény pro hlavní aplikaci a API.
- Musí komunikovat s externími službami nebo widgety hostovanými na jiné doméně.

Výzvy multidoménového testování

- **Politika stejného původu:**

Prohlížeče omezují JavaScript v přístupu ke zdrojům na různých doménách. To může znesnadnit testování interakcí mezi stránkami na různých doménách.

- **Správa relace:**

Udržení a ověření stavu relace napříč doménami může být problematické, protože cookies a localStorage nejsou mezi doménami standardně sdílené.

- **Konzistentní tok dat:**

Zajištění správného toku dat mezi doménami (např. když přihlášení na jedné doméně umožní přístup na druhé) vyžaduje speciální přístup v testech.

C. Pokročilé příkazy Cypress: cy.origin a cy.session

1. cy.origin: Práce s více doménami

- **Účel:**

cy.origin umožňuje spouštět příkazy v kontextu jiné domény (originu). To je užitečné, když vaše aplikace pracuje s více doménami (například ověřování pomocí třetí strany).

- **Příklad použití:**

```
// Příklad: Navigace na externí doménu kvůli nějaké akci
cy.origin('https://external-domain.com', () => {
  cy.get('[data-testid="external-login"]').click();
});
```

- **Výhody:**

Umožňuje testování interakcí mezi doménami při zachování bezpečnostních hranic.

Důležitost cy.origin() při multidoménovém testování

Co je cy.origin() ?

cy.origin() je Cypress příkaz, který vám umožňuje spouštět příkazy v kontextu jiné domény než je váš primární baseUrl. To je zásadní pro testování multidoménových interakcí.

Proč je to důležité:

- **Obcházení omezení stejného původu:**

Umožňuje interagovat s elementy nebo stránkami hostovanými na jiné doméně. Bez cy.origin() by Cypress vyhazoval chyby při pokusu o přístup ke zdrojům na jiné doméně.

- **Realistická uživatelská interakce:**

Když je uživatel přesměrován nebo interaguje s jinou doménou (například platební brána nebo OAuth poskytovatel), cy.origin() vám umožní tyto interakce v testu simulovat.

- **Plynulá integrace:**

Můžete přepínat mezi doménami v rámci jediného testu, což umožní ověřit celou uživatelskou cestu napříč různými částmi vaší aplikace.

Ukázka použití `cy.origin()` :

Představte si, že vaše aplikace vyžaduje ověření přes službu třetí strany na

`https://auth.example.com` , zatímco hlavní aplikace běží na `http://localhost:3000` .

```
// Ve vašem test souboru pro multidoménové přihlášení
describe('Multidomain Login Test', () => {
  it('should handle login via a third-party authentication service', () => {
    // Navštivte hlavní stránku aplikace
    cy.visit('/login');

    // Předpoklad: tlačítko přesměrovává na externí doménu pro ověření
    cy.get('[data-testid="login-with-oauth"]').click();

    // Použijte cy.origin() pro přepnutí kontextu na externí doménu
    cy.origin('https://auth.example.com', () => {
      // Práce s ověřovací stránkou
      cy.get('[data-testid="auth-username"]').type('externalUser');
      cy.get('[data-testid="auth-password"]').type('externalPass');
      cy.get('[data-testid="auth-submit"]').click();
    });

    // Vrátit se na hlavní doménu a ověřit úspěšné přihlášení
    cy.url().should('include', '/dashboard');
    cy.get('[data-testid="welcome-message"]').should('contain', 'Welcome');
  });
});
```

Další příklady pro multidoménové testování

Příklad: Přepínání mezi více doménami

Představte si test, který potřebuje po přihlášení navštívit marketingový web na jiné doméně:

```
describe('Multidomain Navigation', () => {
  it('should navigate from the main app to the marketing site', () => {
    cy.visit('/login');
    cy.get('[data-testid="login-with-oauth"]').click();

    cy.origin('https://auth.example.com', () => {
      cy.get('[data-testid="auth-username"]').type('externalUser');
      cy.get('[data-testid="auth-password"]').type('externalPass');
      cy.get('[data-testid="auth-submit"]').click();
    });

    // Po přihlášení je uživatel přesměrován na marketingovou stránku na další doménu
    cy.origin('https://marketing.example.com', () => {
      cy.visit('/special-offers');
      cy.get('[data-testid="offer-banner"]').should('be.visible');
    });
  });
});
```

D. Aktivita

1. Nakonfigurujte baseUrl a aktualizujte testy:

- **Úkol:**

Upravit testy tak, aby používaly `cy.visit('/')` s nastaveným `baseUrl` v `cypress.config.js`.

- **Cíl:**

Ověřit, že testy používají relativní URL a že se správně aplikuje base URL dle prostředí.

2. Experimentujte s `cy.origin` a `cy.session`:

- **Úkol:**

Napsat test, který přechází na jinou doménu pomocí `cy.origin` (simulace externí autentizace).

- **Cíl:**

Ukázat práci s multidoménovými scénáři a snížit režii nastavení.

E. Ukázky příkazů a konfigurace Cypress

Použití `baseUrl` v testu:

```
describe('Home Page Navigation', () => {
  it('should load the login page using relative URL', () => {
    cy.visit('/login'); // Rozřeší se jako baseUrl + /login
    cy.get('[data-testid="page-title"]').should('contain', 'Login');
  });
});
```

Použití cy.origin:

```
describe('Cross-Origin Login', () => {
  it('should perform login on external domain', () => {
    cy.origin('https://external.example.com', () => {
      cy.get('[data-testid="external-login"]').click();
    });
  });
});
```

Možné dotazy studentů a odpovědi

1. **Q:** *Jak nám baseUrl pomáhá zjednodušit naše testy?*

A: Nastavením baseUrl v konfiguraci můžete v `cy.visit()` používat relativní cesty, což vede k přehlednějším testům a snadnější správě při přepínání prostředí.

2. **Q:** *Co jsou kontexty v Cypressu a proč jsou užitečné?*

A: Kontexty, implementované zanořenými bloky `describe()`, umožňují logicky seskupovat testy (např. podle uživatelských scénářů). Pomáhají udržet vaši testovací sadu organizovanou a izolovanou, což usnadňuje údržbu a ladění.

3. **Q:** *K čemu slouží cy.origin?*

A: `cy.origin` vám umožňuje spouštět příkazy v rámci jiné domény. To je užitečné při testování interakcí mezi doménami, například při ověření pomocí třetí strany.

4. **Q:** *Mohu přepsat baseUrl pro konkrétní testy?*

A: Zatímco baseUrl je globální nastavení, v jednotlivých testech jej můžete přepsat pomocí absolutních URL v `cy.visit()`. Nejlepší je však používat globální baseUrl kvůli konzistenci, ledaže byste měli konkrétní důvod postupovat jinak.

5. **Q:** *Jaké jsou osvědčené postupy pro používání kontextů v Cypressu?*

A: Organizujte testy do zanořených bloků `describe()` s jasnými a výstižnými názvy. Používejte hooky (`beforeEach` / `afterEach`) pro nastavení nebo čištění stavů v rámci každého kontextu, abyste zajistili izolaci a udržitelnost testů.

Častý problém s cross-domain

```
describe('Cypress lost value', () => {
  var value1
  var value2;

  it('10 - navigate and login to domain1 and save value to variable', () => {
    cy.visit('https://domain1.com');
    cy.loginDomain1().then((value) => {
      value1 = value
    })
    value2 = 100
  });

  it('20 - navigate and login to domain2 and with value1 variable', () => {
    cy.visit('https://domain2.com');
    cy.log(value2)
    cy.loginDomain2(value1)
  })
});
```

Proč Cypress všechno resetuje při změně domény?

To je dáno **bezpečnostními pravidly webového prohlížeče**:

Co se vlastně děje?

Když uděláte něco jako:

```
cy.visit('https://domain1.com');
// něco proveďte...
cy.visit('https://domain2.com');
// test selže nebo proměnné zmizí...
```

Cypress **automaticky resetuje celý kontext prohlížeče**, pokud se změní doména.

◆ Proč?

Kvůli **politice stejného původu (SOP)** v prohlížečích:

- JavaScript z jednoho původu (`https://domain1.com`) nemůže přímo interagovat s obsahem na jiném původu (`https://domain2.com`).
- Cypress toto omezení striktně dodržuje, aby ochránil **konzistenci vašich testů** a zamezil úniku dat nebo scope napříč doménami.

Když Cypress detekuje změnu původu:

- **Resetuje testovací iframe**
- Všechny **proměnné v paměti**, DOM, cookies a JavaScriptový scope jsou smazány
- To zajišťuje izolaci testů a bezpečnost

Příklad: Proč je proměnná při změně domény `undefined`

```
let token;

it('gets token from domain1', () => {
  cy.visit('https://domain1.com');
  cy.getCookie('authToken').then((cookie) => {
    token = cookie.value;
  });
});

it('tries to use token on domain2', () => {
  cy.visit('https://domain2.com');
  cy.log(token); // ● token je zde nedefinovaný
});
```

Jaký je problém?

- Proměnná `token` byla deklarována **v hlavním scope testovacího souboru**.
- Když Cypress přejde na novou doménu, vyčistí kontext vykonávání.
- Takže `token` je ztracen — Cypress začíná z čistého stavu.

V Cypressu každý test (`it` blok) běží izolovaně, což znamená, že proměnné nastavené v jednom testu nejsou zaručeně přeneseny do jiného. I když deklarujete globální proměnnou jako `var value1`, Cypress mezi testy resetuje stav právě kvůli testovací izolaci a abyste neměli mezi testy skryté závislosti. Navíc jsou Cypress příkazy asynchronní, takže přiřazení do `value1` v prvním testu nemusí být v době spuštění druhého testu zcela doručeno, nebo mezitím resetováno.

Klíčové body:

- **Izolace testů:**

Cypress spouští každý `it` blok izolovaně, aby zabránil vedlejším efektům mezi testy. Globální proměnné nejsou garantovány mezi testy.

- **Asynchronní příkazy:**

Callback `.then()` v prvním testu přiřadí `value1`, ale když začne druhý test, mechanismus izolace Cypressu stáhne stav zpět.

- **Ověřená praxe:**

Namísto spoléhání na globální proměnné mezi testy využívejte Cypress příkazy jako `cy.session()`, nebo řetězte příkazy v rámci jednoho testu. Pokud potřebujete sdílet data, uložte je do kontextu testu pomocí closure, nebo využijte vlastní úkoly v Cypressu.

Vysvětlení příkladu:

```
describe('Cypress lost value', () => {
  var value1;

  it('10 - navigate and login to domain1 and save value to variable', () => {
    cy.visit('https://domain1.com');
    cy.loginDomain1().then((value) => {
      value1 = value // přiřazuje se zde
    });
    value2 = 100 // přiřazuje se zde
  });

  it('20 - navigate and login to domain2 and with value1 variable', () => {
    cy.visit('https://domain2.com');
    // Při běhu tohoto testu jsou value1, value2 nedefinované, protože izolace testu je resetov:
    cy.log(value2) //-> value2 je nedefinovaná
    cy.loginDomain2(value1); //-> value1 je nedefinovaná
  });
});
```

Ve druhém testu je `value1` `undefined`, protože:

- **Izolace testů:** Každý `it` blok je izolovaný.
- **Asynchronní povaha:** I kdyby byla `value1` nastavena, kvůli asynchronnímu provedení a resetu stavu mezi testy nemusí být již dostupná.

Díky pochopení izolace testů a asynchronní povahy příkazů Cypressu můžete psát testy, které se nespolehají na sdílené externí proměnné.

Pokud potřebujete sdílet hodnotu mezi testy – zvláště mezi různými doménami – je důležité si uvědomit, že Cypress úmyslně izoluje každý blok `it()`. Tato izolace zajišťuje nezávislost testů, ale znamená to, že proměnné nastavené v jednom testu nejsou dostupné v jiném. Zde je několik osvědčených postupů a řešení, jak tuto situaci řešit:

Ověřené postupy pro sdílení stavu mezi testy

1. Držte testy nezávislé:

Ideálně by každý test měl být nezávislý. Pokud potřebujete použít hodnotu ze vstupu na jedné doméně v další, spojte příslušné kroky do jednoho testu, abyste zajistili dostupnost hodnoty.

2. Spojte navazující kroky:

Pokud obě domény tvoří jeden uživatelský tok (např. přihlášení na `domain1` a následná akce na `domain2`), dejte je do jednoho testu. Tím má vše potřebné v daném bloku.

3. Ukládání dat pomocí `cy.task()`:

Můžete využít vlastní úkol (`task`) pro ukládání dat (například zápis do souboru nebo do paměti serveru), které budou přístupné mezi testy i mezi doménami.

Příklad řešení

Řešení 1: Spojení do jednoho testovacího bloku

Pokud uživatelský tok vede z domény1 na doménu2, spojte je:

```
describe('Multidomain Login Flow', () => {
  it('logs in on domain1 and uses the value on domain2', () => {
    // Krok 1: Navštivte domain1 a získejte hodnotu
    cy.visit('https://domain1.com');
    cy.loginDomain1().then((value) => {
      // Hodnota je zde dostupná
      // Krok 2: Přepněte se na doménu2 pomocí cy.origin()
      cy.visit('https://domain2.com');
      cy.origin('https://domain2.com', () => {
        // Použít získanou hodnotu na domain2
        cy.loginDomain2(value).should('succeed');
      });
    });
  });
});
```

Řešení 2: Použití `cy.task()` pro uchování dat mezi testy

Pokud nelze testy spojit, použijte vlastní úkol k uložení hodnoty mimo kontext testu:

1. Definujte task v cypress.config.js nebo samostatném souboru:

```
// V cypress.config.js nebo v podpora souboru
module.exports = defineConfig({
  e2e: {
    setupNodeEvents(on, config) {
      let sharedValue; // Úložiště v paměti

      on('task', {
        saveValue(value) {
          sharedValue = value;
          return null;
        },
        getValue() {
          return sharedValue;
        }
      });
      return config;
    }
  }
});
```

2. Použijte cy.task() v testech:

```
describe('Multidomain Flow with Persisted Value', () => {
  it('saves a value from domain1', () => {
    cy.visit('https://domain1.com');
    cy.loginDomain1().then((value) => {
      cy.task('saveValue', value);
    });
  });

  it('retrieves the value and uses it on domain2', () => {
    cy.visit('https://domain2.com');
    cy.task('getValue').then((value) => {
      // Teď použijte hodnotu na domain2
      cy.loginDomain2(value).should('succeed');
    });
  });
});
```

Řešení 3: Ukládání do souborů

1. Definujte tasks ve funkci `setupNodeEvents` :

Ve vašem `cypress.config.js` (nebo souboru, který je importován) nastavte tasky pro zápis do a čtení ze souboru. Například:

```
const fs = require('fs');
const path = require('path');

module.exports = defineConfig({
  e2e: {
    setupNodeEvents(on, config) {
      // Task pro zápis hodnoty do souboru
      on('task', {
        writeValue({ filename, value }) {
          const filePath = path.join(__dirname, filename);
          fs.writeFileSync(filePath, JSON.stringify(value));
          return null;
        },
        // Task pro čtení hodnoty ze souboru
        readValue({ filename }) {
          const filePath = path.join(__dirname, filename);
          if (fs.existsSync(filePath)) {
            const data = fs.readFileSync(filePath, 'utf8');
            return JSON.parse(data);
          }
          return null;
        }
      });
      return config;
    },
    baseUrl: 'http://localhost:3000'
  },
  env: {
    // Proměnné specifické pro prostředí...
  }
});
```

2. Použití v testech:

Poté můžete použít `cy.task('writeValue', { filename, value })` v jednom testu pro uložení hodnoty a `cy.task('readValue', { filename })` v jiném testu k jejímu načtení.

Příklad:

```

describe('Store and Retrieve Value from File', () => {
  it('should save a value from domain1 to a file', () => {
    cy.visit('https://domain1.com');
    // Předpoklad: cy.loginDomain1() vrací hodnotu, kterou chcete uložit
    cy.loginDomain1().then((value) => {
      // Uložení hodnoty do souboru "sharedValue.json"
      cy.task('writeValue', { filename: 'cypress/sharedValue.json', value });
    });
  });

  it('should retrieve the value from file and use it on domain2', () => {
    cy.visit('https://domain2.com');
    cy.task('readValue', { filename: 'cypress/sharedValue.json' }).then((value) => {
      // Použijte získanou hodnotu, neměla by být undefined
      cy.loginDomain2(value).should('succeed');
    });
  });
});

```

Výhody ukládání do souboru:

- **Perzistence:**

Data přetrvávají i napříč spuštěními nebo v oddělených procesech.

- **Ladění:**

Soubor můžete manuálně prozkoumat a zjistit, jaká data byla uložena.

- **Decoupling:**

Odděluje testy od paměťového úložiště, což je užitečné při paralelním běhu testů nebo při potřebě sdílení dat mezi oddělenými testovacími sadami.

Nevýhody:

- **Zátěž I/O:**

Operace se soubory jsou obvykle pomalejší než operace v paměti.

- **Komplexita:**

Správa souborového úložiště a úklid může přinést do testovací sady další složitost.

- **Možná nestabilita:**

Pokud souborové operace selžou nebo když testy běží paralelně a pokouší se přistupovat ke stejnému souboru, může nastat race condition. Správná izolace a úklid jsou klíčové.

Ověřená praxe:

- **Unikátní názvy souborů:**

Používejte unikátní názvy souborů pro jednotlivé testovací sady, abyste zabránili konfliktům.

- **Čištění:**

Po spuštění testů soubory smažte nebo resetujte, abyste se vyhnuli starým datům.

- **Zpracování chyb:**

Ujistěte se, že vaše tasky správně zachycují chyby (např. nenalezený soubor), aby testy mohly selhat rozumně.

Ukládání do souboru pomocí `cy.task()` je robustní řešení pro uchování hodnot napříč testy, obzvlášť když jiné metody (jako globální proměnné či `cy.session()`) nejsou vhodné. Je to zvlášť užitečné pro multidoménové nebo multi-session scénáře, kdy potřebujete sdílet data externě.